Campusmonk

TCS PYQ

Maximum Points You Can Obtain from Cards

String Encryption

Campusmonk

Maximum Points You Can Obtain from Cards

Given N cards arranged in a row, each card has an associated score denoted by the cardScore array. Choose exactly k cards. In each step, a card can be chosen either from the beginning or the end of the row. The score is the sum of the scores of the chosen cards.

Return the **maximum score** that can be obtained.

Examples:

Input : cardScore = [1, 2, 3, 4, 5, 6] , k = 3

Output : 15

Explanation : Choosing the rightmost cards will maximize your total score. So optimal cards chosen are the rightmost three cards 4 , 5 , 6.

Th score is 4 + 5 + 6 => 15.

Input : cardScore = [5, 4, 1, 8, 7, 1, 3] , k = 3

Output : 12

Campusmonk

Campusmonk

Campusmonk

Campusmonk

Main.java

```java
import java.util.*;
class Main{
    public static int maxScore(int[] cardScore, int k) {
        int lSum = 0, rSum = 0, maxSum = 0;
        for (int i = 0; i < k; i++) {
            lSum += cardScore[i];
            maxSum = lSum;
        }
        int rightIndex = cardScore.length - 1;

        for (int i = k - 1; i >= 0; i--) {
            lSum -= cardScore[i];
            rSum += cardScore[rightIndex];
            rightIndex--;
            maxSum = Math.max(maxSum, lSum + rSum);
        }
        return maxSum;
    }

    public static void main(String[] args) {
        int[] nums = {1, 2, 3, 4, 5, 6};
        int result = maxScore(nums, 3);

        System.out.println("The maximum score is:");
        System.out.println(result);
    }
}
```

Campusmonk

main.cpp

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int maxScore(vector<int>& cardScore, int k) {
    int lSum = 0, rSum = 0, maxSum = 0;

    for (int i = 0; i < k; i++) {
        lSum += cardScore[i];
    }
    maxSum = lSum;

    int rightIndex = cardScore.size() - 1;

    for (int i = k - 1; i >= 0; i--) {
        lSum -= cardScore[i];
        rSum += cardScore[rightIndex];
        rightIndex--;
        maxSum = max(maxSum, lSum + rSum);
    }

    return maxSum;
}

int main() {
    vector<int> nums = {1, 2, 3, 4, 5, 6};
    int result = maxScore(nums, 3);
    cout << "The maximum score is:" << endl;
    cout << result << endl;
    return 0;
}
```

Campusmonk

Q1. The Caesar cipher is a type of substitution cipher in which each alphabet in the plaintext or messages is shifted by a number of places down the alphabet.
For example,with a shift of 1, P would be replaced by Q, Q would become R, and so on.
To pass an encrypted message from one person to another, it is first necessary that both parties have the ‘Key’ for the cipher, so that the sender may encrypt and the receiver may decrypt it.
Key is the number of OFFSET to shift the cipher alphabet. Key can have basic shifts from 1 to 25 positions as there are 26 total alphabets.
As we are designing custom Caesar Cipher, in addition to alphabets, we are considering numeric digits from 0 to 9. Digits can also be shifted by key places.
For Example, if a given plain text contains any digit with values 5 and key =2, then 5 will be replaced by 7, “-”(minus sign) will remain as it is. Key value less than 0 should result into “INVALID INPUT”

Example 1:
Enter your PlainText: All the best
Enter the Key: 1

The encrypted Text is: Bmm uif Cftu

Campusmonk

Campusmonk

Campusmonk

Campusmonk

main.cpp

Share | Run

Output

```
#include <iostream>
#include <string>
using namespace std;
int main() {
    string str;
    int key, i = 0, left;
    cout << "Enter your plain text: ";
    getline(cin, str);
    cout << "Enter the key: ";
    cin >> key;
    if (key == 0) {
        cout << "INVALID INPUT" << endl;
    } else {
        while (str[i] != '\0') {
            if (str[i] >= '0' && str[i] <= '9') {
                if (str[i] + key <= '9') {
                    str[i] = str[i] + key;
                } else {
                    left = (str[i] + key) - '9';
                    str[i] = '0' + left - 1;
                }
            } else if (str[i] >= 'A' && str[i] <= 'Z') {
                if (str[i] + key <= 'Z') {
                    str[i] = str[i] + key;
                } else {
                    left = (str[i] + key) - 'Z';
                    str[i] = 'A' + left - 1;
                }
            } else if (str[i] >= 'a' && str[i] <= 'z') {
                if (str[i] + key <= 'z') {
                    str[i] = str[i] + key;
                } else {
                    left = (str[i] + key) - 'z';
                    str[i] = 'a' + left - 1;
                }
            }
            i++;
        }
        cout << "The encrypted text is: " << str << endl;
    }
    return 0;
}
```

```
/tmp/DhsFLvdSOx.o
Enter your plain text: Hello123
Enter the key: 4
The encrypted text is: Lipps567


=== Code Execution Successful ===
```

Campusmonk

Main.java

Share Run

Output

```
java -cp /tmp/WwCSJpqpUG/CaesarCipher
Enter your plain text: Hello123
Enter the key: 4
The encrypted text is: Lipps567
```

```java
import java.util.Scanner;
public class CaesarCipher {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter your plain text: ");
        String str = sc.nextLine();
        System.out.print("Enter the key: ");
        int key = sc.nextInt();
        if (key == 0) {
            System.out.println("INVALID INPUT");
        } else {
            StringBuilder encryptedText = new StringBuilder();
            for (int i = 0; i < str.length(); i++) {
                char ch = str.charAt(i);
                if (ch >= '0' && ch <= '9') {
                    // For digits '0' to '9'
                    ch = (char) (ch + key);
                    if (ch > '9') {
                        int left = (ch - '9');
                        ch = (char) ('0' + left - 1);
                    }
                } else if (ch >= 'A' && ch <= 'Z') {
                    ch = (char) (ch + key);
                    if (ch > 'Z') {
                        int left = (ch - 'Z');
                        ch = (char) ('A' + left - 1);
                    }
                } else if (ch >= 'a' && ch <= 'z') {
                    ch = (char) (ch + key);
                    if (ch > 'z') {
                        int left = (ch - 'z');
                        ch = (char) ('a' + left - 1);
                    }
                }
                encryptedText.append(ch);
            }
            System.out.println("The encrypted text is: " +
                encryptedText.toString());
        }
        sc.close();
    }
}
```

Campusmonk